

TRABAJO PRACTICO n-2

LAUTARO SUAREZ

INTRODUCCIÓN

Ejercicio 1: Se desea conocer el estado de la solicitud del préstamo (OTORGADO o RECHAZADO) para personas de entre 40 y 45 años a partir de los atributos categóricos.

Registro de datos

- Dividir aleatoriamente el conjunto de datos para utilizar una parte de los mismos como conjunto de entrenamiento (el 80 %) y otro como conjunto de prueba (el 20 %).

- Implementar el algoritmo ID3, utilizando la entropía de Shannon para la función de ganancia de información, para clasificar los ejemplos del conjunto de prueba.

- Calcular el accuracy y el F1-score.

- Construir la matriz de confusión para el conjunto de prueba.



PREPARACIÓN DEL ENTORNO

Configuramos el entorno de Python y cargamos el dataset de préstamos desde Google Drive para iniciar el análisis

```
1 import pandas as pd
2 import numpy as np
3 import math
4 from google.colab import drive
5 import math
6 import json
7 import matplotlib.pyplot as plt
8 drive.mount('/content/drive')
```

```
ruta = '/content/drive/MyDrive/A_A/Préstamo.csv'

df = pd.read_csv(ruta, sep=',', encoding='latin-1')
```

FILTRADO DE DATOS

Seleccionamos las columnas solicitadas

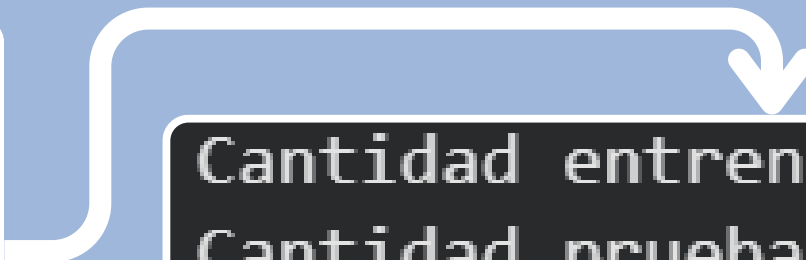
Clasificamos la muestra en el rango de 40 a 45 años, obteniendo un total de 1426 registros

```
misColumnas = 'Edad','Sexo','Mayor nivel educativo','Estado de vivienda',  
'Préstamos previos impagos','Estado'  
df1 = df.loc[ : ,misColumnas]  
df_40_45 = df1[(df1["Edad"] >=40) & (df1["Edad"] <= 45) ]  
df_40_45 = df_40_45.drop('Edad', axis=1)  
print("cantidad de registros",df_40_45['Sexo'].count())
```

cantidad de registros 1426

FILTRADO DE DATOS

```
df_40_45 = df_40_45.sample(frac=1, random_state=42).reset_index(drop=True)
n = int(len(df_40_45) * 0.8)
df_40_45_entrenamiento = df_40_45.iloc[:n]
df_40_45_test = df_40_45.iloc[n:]
print('Cantidad entrenamiento:', len(df_40_45_entrenamiento))
print('Cantidad prueba:', len(df_40_45_test))
```



Cantidad entrenamiento: 1140
Cantidad prueba: 286

Mezclamos los datos aleatoriamente para eliminar sesgos de orden y asegurar que el aprendizaje del modelo sea equilibrado y reproducible

Dividimos la muestra en conjuntos de entrenamiento (1140) y de prueba (286) para validar la efectividad de las futuras predicciones.

FILTRADO DE DATOS

```
conteo = df_40_45_entrenamiento['Estado'].value_counts()  
for clase in conteo.index:  
    cantidad = conteo[clase]  
    print(clase, cantidad)
```



```
RECHAZADO 904  
OTORGADO 236
```

Podemos observar una clara diferencia con 904 solicitudes rechazadas y solo 236 otorgadas

Los datos reflejan que el modelo tendrá muchos más ejemplos de préstamos rechazados que otorgados para aprender durante el entrenamiento

ENTROPÍA Y GANANCIA DE INFORMACIÓN

Calculamos primero la entropía total del conjunto de entrenamiento y luego la compara con la entropía ponderada de cada variable. Al restar ambos valores, identificamos el atributo con la mayor ganancia de información para liderar nuestro modelo

```
def entropiaShanon(df, columna_clase):
    conteo = df[columna_clase].value_counts()
    total = len(df)
    entropia_total = 0
    for cantidad in conteo:
        p = cantidad / total
        entropia_total += -p * math.log2(p)
    max_ganancia = -1
    max_atributo = None
    for atributo in df.columns:
        if atributo != columna_clase:
            entropia_atributo = 0
            conteo_atributo = df[atributo].value_counts()
            for categoria in conteo_atributo.index:
                subset = df[df[atributo] == categoria]
                total_categoria = len(subset)
                entropia_categoria = 0
                conteo_clases = subset[columna_clase].value_counts()
```


ENTROPÍA Y GANANCIA DE INFORMACIÓN

"Préstamos previos impagos" resultó ser el atributo con mayor ganancia de información. Esto significa que es la variable más determinante para predecir si un préstamo será otorgado o rechazado

```
for etiqueta in conteo_clases.index:
    cantidad = conteo_clases[etiqueta]
    p = cantidad / total_categoria
    entropia_categoria += -p * math.log2(p)
peso = total_categoria / total
entropia_atributo += peso * entropia_categoria
ganancia = entropia_total - entropia_atributo
if ganancia > max_ganancia:
    max_ganancia=ganancia
    max_atributo=atributo
return max_atributo

entropiaShanon(df_40_45_entrenamiento,'Estado')
```

"Préstamos previos impagos"

CONSTRUCCIÓN RECURSIVA DEL ÁRBOL

```
def construir_arbol(df, columna_clase):
    if df.empty:
        return None
    if df[columna_clase].nunique() == 1:
        return {
            'resultado': df[columna_clase].iloc[0]
        }
    if len(df.columns) == 1:
        return {
            'resultado': df[columna_clase].mode()[0]
        }
    mejor_atributo = entropiaShanon(df, columna_clase)
    arbol = {
        'atributo': mejor_atributo,
        'ramas': {}
    }
    for categoria in df[mejor_atributo].unique():
        subset = df[df[mejor_atributo] == categoria]
        subset = subset.drop(mejor_atributo, axis=1)
        subarbol = construir_arbol(subset, columna_clase)
        arbol['ramas'][categoria] = subarbol
    return arbol
```

Esta función define la estructura del árbol mediante tres reglas clave: frena si los datos son puros, asigna la mayoría si no quedan variables, o crea un nuevo nodo ramificando los datos según las categorías del atributo más determinante.

VISUALIZACIÓN DE LA ESTRUCTURA DEL ÁRBOL

```
def mostrar_arbol(arbol, nivel=0):  
    if 'resultado' in arbol:  
        print(' ' * nivel + '→', arbol['resultado'])  
        return  
    print(' ' * nivel + '[' + arbol['atributo'] + ']')  
    for categoria, subarbol in arbol['ramas'].items():  
        print(' ' * (nivel + 1) + categoria)  
        mostrar_arbol(subarbol, nivel + 2)
```

Implementamos una función para visualizar la estructura del árbol de decisión en formato de texto. Utiliza recursividad y sangrías para mostrar claramente cómo se ramifican los atributos según sus categorías hasta llegar a la resolución de otorgado o rechazado.



```
arbol = construir_arbol(df_40_45_entrenamiento, 'Estado')  
mostrar_arbol(arbol)
```

ESTRUCTURA DEL ÁRBOL

Préstamos previos impagos (SI) es el filtro más drástico: deriva directamente en un rechazo

```
[Préstamos previos impagos]
SI
  → RECHAZADO
NO
  [Estado de vivienda]
  HIPOTECA
    [Mayor nivel educativo]
    Terciario
      [Sexo]
      Masculino
        → RECHAZADO
      Femenino
        → RECHAZADO
    Secundario
      [Sexo]
      Masculino
        → RECHAZADO
      Femenino
        → RECHAZADO
    Doctorado
      [Sexo]
      Masculino
        → RECHAZADO
      Femenino
        → OTORGADO
    Maestría
      [Sexo]
      Masculino
        → RECHAZADO
      Femenino
        → RECHAZADO
    Universitario
      [Sexo]
      Femenino
        → RECHAZADO
      Masculino
        → RECHAZADO
```



```
MAESTRÍA
[Sexo]
Femenino
  → RECHAZADO
Masculino
  → OTORGADO
PROPIETARIO
[Mayor nivel educativo]
Terciario
[Sexo]
Masculino
  → RECHAZADO
Femenino
  → RECHAZADO
Maestría
  → RECHAZADO
Secundario
[Sexo]
Masculino
  → RECHAZADO
Femenino
  → RECHAZADO
Universitario
  → RECHAZADO
Doctorado
[Sexo]
Masculino
  → OTORGADO
OTRO
[Mayor nivel educativo]
Secundario
  → OTORGADO
Terciario
[Sexo]
Femenino
  → OTORGADO
Masculino
  → OTORGADO
Universitario
  → RECHAZADO
```

PREDICCIÓN

```
def predecir_fila(arbol, fila):  
    if 'resultado' in arbol:  
        return arbol['resultado']  
    atributo = arbol['atributo']  
    valor_fila = fila[atributo]  
    if valor_fila in arbol['ramas']:  
        return predecir_fila(arbol['ramas'][valor_fila], fila)  
    else:  
        return 'RECHAZADO'  
  
y_pred = df_40_45_test.apply(lambda fila: predecir_fila(arbol, fila), axis=1)  
df_40_45_test = df_40_45_test.copy()  
df_40_45_test['Prediccion'] = y_pred  
print(df_40_45_test[['Estado', 'Prediccion']].head(10))
```



Estado	Prediccion
RECHAZADO	OTORGADO
OTORGADO	RECHAZADO
RECHAZADO	RECHAZADO
RECHAZADO	OTORGADO
RECHAZADO	RECHAZADO
RECHAZADO	RECHAZADO
RECHAZADO	RECHAZADO
RECHAZADO	RECHAZADO
RECHAZADO	RECHAZADO
RECHAZADO	RECHAZADO

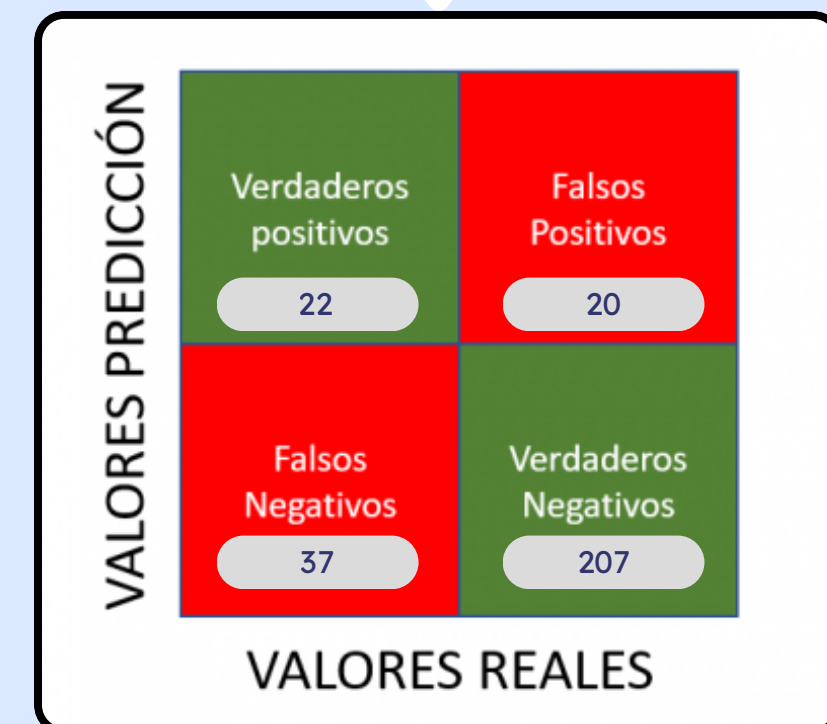
Aplicamos el árbol entrenado sobre el 20% de los datos reservados para testeo. El sistema genera una nueva columna de predicciones para contrastar directamente las decisiones del modelo contra la realidad.

MATRIZ DE CONFUSIÓN

Creamos una función para calcular la Matriz de Confusión

El resultado en el set de prueba revela que el modelo identificó correctamente 207 préstamos rechazados (TN) y 22 otorgados (TP), cometiendo 20 errores de falsos positivos y 37 de falsos negativos

```
def evaluar_modelo(y_real, y_pred):  
    TP = 0  
    TN = 0  
    FP = 0  
    FN = 0  
    for real, pred in zip(y_real, y_pred):  
        if real == 'OTORGADO' and pred == 'OTORGADO':  
            TP += 1  
        elif real == 'RECHAZADO' and pred == 'RECHAZADO':  
            TN += 1  
        elif real == 'RECHAZADO' and pred == 'OTORGADO':  
            FP += 1  
        elif real == 'OTORGADO' and pred == 'RECHAZADO':  
            FN += 1
```



A diagram of a confusion matrix. It consists of a 2x2 grid of colored squares. The top-left square is green and labeled 'Verdaderos positivos' with the value 22. The top-right square is red and labeled 'Falsos Positivos' with the value 20. The bottom-left square is red and labeled 'Falsos Negativos' with the value 37. The bottom-right square is green and labeled 'Verdaderos Negativos' with the value 207. To the left of the grid, the text 'VALORES PREDICCIÓN' is written vertically. Below the grid, the text 'VALORES REALES' is written horizontally. A white arrow points from the code block above to the matrix.

VALORES PREDICCIÓN \ VALORES REALES	OTORGADO	RECHAZADO
OTORGADO	Verdaderos positivos 22	Falsos Positivos 20
RECHAZADO	Falsos Negativos 37	Verdaderos Negativos 207

ACCURACY / F1-SCORE

```
total = TP + TN + FP + FN
accuracy = (TP + TN) / total if total > 0 else 0

precision = TP / (TP + FP) if (TP + FP) > 0 else 0
recall = TP / (TP + FN) if (TP + FN) > 0 else 0

f1_score = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0
```

Métricas de Evaluación
Accuracy: 0.8007
F1-Score: 0.4356

El modelo acierta en 8 de cada 10 predicciones, demostrando una alta efectividad general para clasificar el set de prueba

Sin embargo, al evaluar exclusivamente los préstamos "Otorgados", la precisión cae drásticamente. El algoritmo es excelente detectando rechazos, pero le cuesta identificar los casos positivos debido a la falta de ejemplos en el entrenamiento

INTRODUCCIÓN

Ejercicio 2: Utilizando los mismos conjuntos de entrenamiento y de prueba que en el Ejercicio 1, se pide lo siguiente:

Registro de datos

- Aplicar el método de Random Forest con 10 árboles para clasificar los ejemplos del conjunto de prueba.



- Construir la matriz de confusión para el conjunto de prueba.



- Calcular el accuracy y el F1-score.



- Realizar el gráfico de precisión, tanto del conjunto de entrenamiento como del conjunto de prueba, en función del tamaño de cada uno de los árboles del bosque

RANDOM FOREST

```
def entrenar_random_forest(df_train, columna_clase, n_arboles=10, semilla=42):  
    if semilla is not None:  
        np.random.seed(semilla)  
    bosque = []  
    n_filas = len(df_train)  
    for i in range(n_arboles):  
        indices_bootstrap = np.random.choice(n_filas, size=n_filas,  
                                             replace=True)  
        df_bootstrap = df_train.iloc[indices_bootstrap].reset_index(drop=True)  
        arbol = construir_arbol(df_bootstrap, columna_clase)  
        bosque.append(arbol)  
    return bosque
```

Implementamos un modelo de Random Forest desde cero. El algoritmo genera múltiples subconjuntos de datos mediante la técnica de Bootstrap

entrena un árbol de decisión independiente en cada uno de ellos para formar nuestro bosque predictivo

PREDICCIÓN RDM FOREST

El algoritmo recorre el bosque acumulando los votos de cada estructura. Si los votos positivos superan a los negativos, el crédito se aprueba, de lo contrario, se rechaza.

```
def predecir_random_forest(bosque, fila):  
    votos_otorgado = 0  
    votos_rechazado = 0  
    for arbol in bosque:  
        voto = predecir_fila(arbol, fila)  
        if voto == 'OTORGADO':  
            votos_otorgado += 1  
        else:  
            votos_rechazado += 1  
    if votos_otorgado > votos_rechazado:  
        return 'OTORGADO'  
    else:  
        return 'RECHAZADO'
```

COMPARATIVA DE PREDICCIONES

```
mi_bosque = entrenar_random_forest(df_40_45_entrenamiento, 'Estado',
n_arboles=10)
y_pred_bosque = df_40_45_test.apply(lambda fila: predecir_random_forest
(mi_bosque, fila), axis=1)

df_40_45_test['Prediccion_RF'] = y_pred_bosque
print(df_40_45_test[['Estado', 'Prediccion', 'Prediccion_RF']].head(10))
```

Registramos en el dataset las predicciones de ambos algoritmos frente al estado real del préstamo. Esta estructura nos permite preparar el escenario para el cálculo definitivo de las nuevas métricas de evaluación

[illegible]

MÉTRICAS

```
print("EVALUACIÓN RANDOM FOREST")  
evaluar_modelo(df_40_45_test['Estado'], df_40_45_test['Prediccion_RF'])
```

VALORES PREDICCIÓN	VALORES REALES	
	Verdaderos positivos 14	Falsos Positivos 18
Falsos Negativos 45	Verdaderos Negativos 209	

El Random Forest alcanza un 77.97% de exactitud general, mostrando una gran solidez para identificar los préstamos rechazados (209 casos)

Métricas de Evaluación
Accuracy: 0.7797
F1-Score: 0.3077

El F1-Score (0.3077) disminuye en comparación con el árbol único, demostrando que la votación por mayoría volvió al modelo más conservador y estricto ante la clase minoritaria

HISTOGRAMA

```
def calcular_tamano_arbol(arbol):  
    if 'resultado' in arbol:  
        return 1  
    tamaño = 1  
    for rama in arbol['ramas'].values():  
        tamaño += calcular_tamano_arbol(rama)  
    return tamaño
```

```
tamanos_desordenados = []  
acc_train_desordenado = []  
acc_test_desordenado = []
```

```
for arbol in mi_bosque:  
    tamanos_desordenados.append(calcular_tamano_arbol(arbol))  
    pred_train = df_40_45_entrenamiento.apply(lambda fila: predecir_fila(arbol,  
    fila), axis=1)  
    aciertos_tr = sum((real == pred) for real, pred in zip  
    (df_40_45_entrenamiento['Estado'], pred_train))  
    acc_train_desordenado.append(aciertos_tr / len(df_40_45_entrenamiento))  
    pred_test = df_40_45_test.apply(lambda fila: predecir_fila(arbol, fila),  
    axis=1)  
    aciertos_te = sum((real == pred) for real, pred in zip(df_40_45_test  
    ['Estado'], pred_test))  
    acc_test_desordenado.append(aciertos_te / len(df_40_45_test))
```

Implementamos una función para medir la complejidad (tamaño en nodos) de cada estructura del bosque. Luego, evaluamos individualmente el Accuracy de cada árbol en los conjuntos de entrenamiento y prueba para analizar cómo varía el rendimiento según el tamaño.

CURVA DE APRENDIZAJE Y SOBREAJUSTE

Ordenamos los árboles según su cantidad total de nodos para graficar su comportamiento. La curva nos permite contrastar la precisión en entrenamiento (azul) contra la de prueba (rojo), ayudando a detectar visualmente si los árboles más grandes sufren de sobreajuste.

```
datos_ordenados = sorted(zip(tamamos_desordenados, acc_train_desordenado, acc_test_desordenado))
```

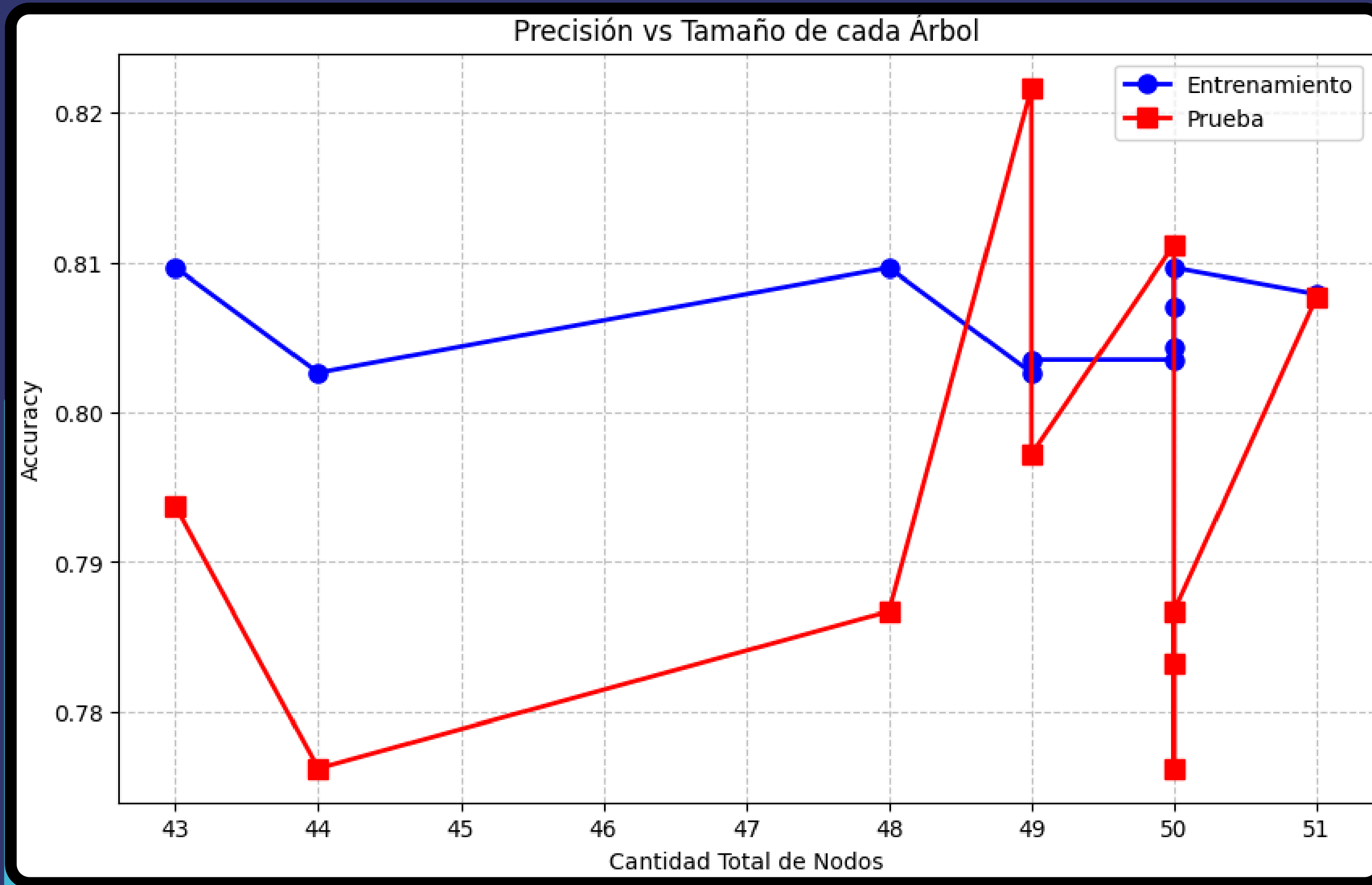
```
tamamos = [d[0] for d in datos_ordenados]  
acc_train = [d[1] for d in datos_ordenados]  
acc_test = [d[2] for d in datos_ordenados]
```

```
plt.figure(figsize=(10, 6))
```

```
plt.plot(tamamos, acc_train, color='blue', label='Entrenamiento', marker='o',  
linestyle='-', linewidth=2, markersize=8)  
plt.plot(tamamos, acc_test, color='red', label='Prueba', marker='s',  
linestyle='-', linewidth=2, markersize=8)
```

```
plt.title('Precisión vs Tamaño de cada Árbol')  
plt.xlabel('Cantidad Total de Nodos')  
plt.ylabel('Accuracy')  
plt.legend()  
plt.grid(True, linestyle='--', alpha=0.7)  
  
plt.show()
```

Registro de datos



INTRODUCCIÓN

Ejercicio 3: ¿Cuál de los métodos se recomienda utilizar para predecir si se otorgaría el crédito bancario solicitado por una persona?
Justificar la respuesta.

Resultado

- Se recomienda utilizar el árbol ID3

- Tras la evaluación, el árbol ID3 simple obtuvo un F1-Score de 0.4356, superando al Random Forest (0.3077). Esto indica que el modelo simple es más capaz de equilibrar la detección de clientes con crédito OTORGADO sin sacrificar la precisión en el rechazo de clientes no aptos

Modelo	Accuracy	F1-Score
ID3 Único	0.8007	0.4356
Random Forest	0.7797	0.3077

- el Random Forest tuvo un desempeño inferior. Esto se debe a que el conjunto de datos es pequeño y presenta una estructura lógica clara que el árbol ID3 logró capturar correctamente.

¡MUCHAS GRACIAS!

